

WATER PLANTS

Company: Google

Round: Online Assessment

Year: 2021

Difficulty: Easy - Medium

Topic: Arrays, Simulation

Source: https://algo.monster/problems/google_oa_water_plants

Problem Description:

Imagine you are filling a row of bowls with soup.

You have a ladle with a fixed capacity, meaning it can hold only a certain amount of soup at one time.

The bowls are arranged in a straight line, one step apart:

Stock Pot → Bowl 0 → Bowl 1 → Bowl 2 → Bowl 3 → ...

You start at the stock pot, which is one step before the first bowl.

You have to follow these rules:

1. Fill the bowls from left to right.
2. You must fill a bowl completely in one go.
3. If the ladle does not have enough soup for the next bowl, go back to the stock pot and refill it.
4. After refilling, return to the same bowl and continue.
5. You may refill even if the ladle is not completely empty.
6. The goal is to calculate the total number of steps taken.

The important point is that every movement matters:

- Moving from one bowl to the next → 1 step.
- Going back to the stock pot → steps equal to the current position.
- Returning to the bowl → the same number of steps again.

Input Format:

We are given:

bowls

An integer array where:

`bowls[i]`

represents the amount of soup needed by bowl i.

`capacity`

The maximum amount of soup the ladle can hold.

For example:

`bowls = [2, 4, 5, 1, 2]`

`capacity = 6`

Output Format:

Return an integer representing the total number of steps needed to fill all the bowls.

Sample Input:

`bowls = [2, 4, 5, 1, 2]`

`capacity = 6`

Sample Output:

Total Number of Steps = 17

Explanation:

Let's go through the example step by step.

`bowls = [2, 4, 5, 1, 2]`

`capacity = 6`

Initially, the ladle contains 6 units of soup.

We start at the stock pot, which is one step before bowl 0.

Bowl 0

Bowl 0 needs 2.

We have 6, so we can fill it.

Remaining soup:

$$6 - 2 = 4$$

Moving to bowl 0 takes:

1 step

Bowl 1

Bowl 1 needs 4.

We have exactly 4, so we can fill it.

Remaining soup:

$$4 - 4 = 0$$

Moving from bowl 0 to bowl 1 takes:

1 step

So far:

2 steps

Bowl 2

Bowl 2 needs 5, but the ladle is empty.

So we must go back to the stock pot.

Bowl 2 is 3 steps away from the stock pot.

Therefore:

3 steps back

Then:

3 steps to return

Now the ladle is full again with 6.

Bowl 2 needs 5.

After filling it:

$$6 - 5 = 1$$

We move to bowl 2, which adds:

1 step

Bowl 3

Bowl 3 needs 1.

We have exactly 1, so we can fill it.

Moving to bowl 3 takes:

1 step

The ladle is now empty again.

Bowl 4

Bowl 4 needs 2.

We don't have enough soup, so we go back to the stock pot.

Bowl 4 is 5 steps away.

So:

5 steps back
+ 5 steps to return

Then we fill bowl 4.

Moving to bowl 4 takes:

1 step

Therefore, the total is:

$$2 + 6 + 2 + 7 = 17$$

So the answer is:

17

Approach to Solve This Problem:

We can solve this problem by simulating the process exactly as it happens.

We keep track of:

- How much soup is currently in the ladle.
- Our current position.
- The total number of steps.

Step 1: Start With a Full Ladle

Initially:

soup = capacity

For the example:

soup = 6

Step 2: Visit Each Bowl From Left to Right

We process the bowls one by one:

$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

For every bowl, we check whether the ladle has enough soup.

Step 3: Check Whether We Need to Refill

Suppose the current bowl needs 5 units and we have only 1.

We cannot partially fill the bowl.

Therefore, we must go back to the stock pot.

If the current bowl is at index i , its distance from the stock pot is:

$i + 1$

So we add:

$2 \times (i + 1)$

to the answer:

- $(i + 1)$ steps to go back.
- $(i + 1)$ steps to come back.

Then we refill the ladle:

soup = capacity

Step 4: Move to the Bowl

Once we have enough soup, we move to the bowl.

Since each bowl is one step away from the previous bowl, moving forward costs:

1 step

Then we fill the bowl:

soup -= bowls[i]

7. Important Observation

There is one small detail that is easy to miss.

Suppose the ladle has some soup left, but it isn't enough for the next bowl.

We still have to refill completely.

For example:

capacity = 6

current soup = 2

next bowl needs = 5

We cannot use the 2 and somehow partially fill the bowl.

We must:

Go back → refill → return → fill the bowl

This is why we check:

if (soup < bowls[i])

before filling each bowl.

Java Code:

```
1 import java.util.*;
2
3 public class Main {
4
5     public static int waterPlants(int[] bowls, int capacity) {
6
7         int soup = capacity;
8         int steps = 0;
9
10        for (int i = 0; i < bowls.length; i++) {
11
12            // Not enough soup for the current bowl
13            if (soup < bowls[i]) {
14
15                // Go back to the stock pot and return
16                steps += 2 * (i + 1);
17
18                // Refill the ladle
19                soup = capacity;
20            }
21
22            // Move to the current bowl
23            steps++;
24
25            // Fill the bowl
26            soup -= bowls[i];
27        }
28
29        return steps;
30    }
31}
```

```
Run | Debug
2 public static void main(String[] args) {
3
4     Scanner sc = new Scanner(System.in);
5
6     int n = sc.nextInt();
7
8     int[] bowls = new Scanner sc - Main.main(String[])
9
10    for (int i = 0; Source: Java IP Session_6389a7e2
11        bowls[i] = sc.nextInt();
12    }
13
14    int capacity = sc.nextInt();
15
16    System.out.println(waterPlants(bowls, capacity));
17
18    sc.close();
19 }
20 }
```

Solution:

The problem can be solved by simply simulating the filling process.

We start with a full ladle and process every bowl from left to right.

For each bowl:

1. Check if the ladle has enough soup.
2. If not, go back to the stock pot and return to the bowl.
3. Refill the ladle.
4. Move to the bowl.
5. Fill it and subtract its required amount from the ladle.

The important formula when we need a refill is:

$$\text{Return trip} = 2 \times (i + 1)$$

because the stock pot is one position before bowl 0.

For the given example, the total number of steps is:

17

Understanding the Important Part of the Code:

1. Start With a Full Ladle

```
int soup = capacity;
```

At the beginning, the ladle is full.

2. Keep Track of Steps

```
int steps = 0;
```

This variable stores the total distance we have travelled.

3. Process Every Bowl

```
for (int i = 0; i < bowls.length; i++)
```

We visit every bowl from left to right.

4. Check if Refill Is Needed

```
if (soup < bowls[i])
```

If the amount of soup currently available is less than what the bowl needs, we must refill.

5. Calculate the Return Trip

steps += 2 * (i + 1);

Why i + 1?

Because bowl 0 is one step away from the stock pot.

So:

Bowl 0 → 1 step

Bowl 1 → 2 steps

Bowl 2 → 3 steps

...

We multiply by 2 because we:

go back + come back

For example, for bowl 2:

$$2 \times (2 + 1)$$

$$= 2 \times 3$$

$$= 6 \text{ steps}$$

6. Refill the Ladle

soup = capacity;

After returning to the stock pot, the ladle becomes full again.

7. Move Forward

steps++;

We move one step to reach the current bowl.

8. Use the Soup

soup -= bowls[i];

After filling the bowl, we subtract the amount of soup it needed.

Complexity Analysis:

Time Complexity: $O(n)$

We visit every bowl exactly once.

The refill operation takes constant time, so the total time is:

Time Complexity = $O(n)$

Space Complexity: $O(1)$

We only use a few variables:

soup
steps
i

No additional data structure is required.

Space Complexity = $O(1)$